parrot ML project

PARROT BEST TEAM

인기있는 기사 예측 프로젝트

Timeline

Process of the machine learning project



1st
Data preparation

Load data, preprocessing, and feature engineering.

2nd
Searching Hyperparameter

Tuning with Optuna.

3rd
Training

5-fold with optimal parameters.

4th
Ensmebling

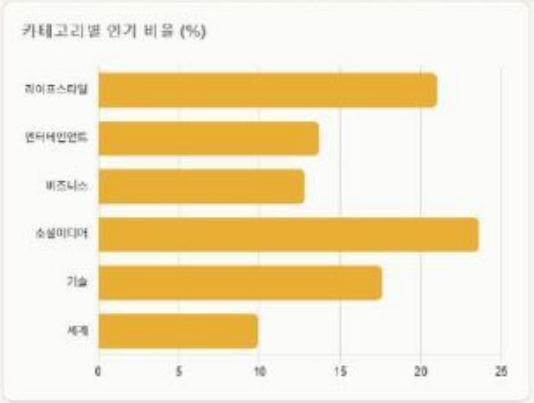
Weight optimization, Stacking.

5th
Submission

Selecting Final Ensemble, Save CSV.

DATA overview







타겟과 상관관계 Top 10 피처

#	피처명	상관계수	방향
1	kw_avg_avg	0.1704	▲ 양의 상관
2	LDA_03	0.1181	▲ 양의 상관
3	LDA_02	-0.1096	▼ 음의 상관
4	data_channel_is_world	-0.0954	▼ 음의 상관
5	kw_max_avg	0.0881	▲ 양의 상관
6	num_imgs	0.0790	▲ 양의 상관
7	num_hrefs	0.0734	▲ 양의 상관
8	kw_min_avg	0.0733	▲ 양의 상관
9	self_reference_avg_sharess	0.0706	▲ 양의 상관
10	is_weekend	0.0665	▲ 양의 상관

csv 세 파일의 데이터를 몇개의 분류기준으로
간단한 분석 시행 → feature engineering 설계

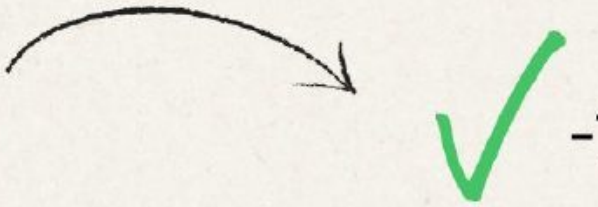
데이터 전처리

```python
# data preprocessing
for df in [train, test, lb]:
    df['kw_min_min'] = df['kw_min_min'].replace(-1, 0)
    df['kw_max_min'] = df['kw_max_min'].replace(-1, 0)

print(f" 전체 기사 중 인기있는 기사(is_popular) 비율: {train['is_popular'].mean():.3f}")
```



일부 변수에서 비정상 값 존재



-1 값을 0으로 치환함

데이터 손실 최소화

Feature Engineering

```python
# feature engineering
def add_features(df): # 기존 59개 + 추가 29개
    df = df.copy()
    df['img_per_word']    = df['num_imgs']   / (df['n_tokens_content'] + 1) #글 길이 대비 이미지 수
    df['video_per_word']  = df['num_videos'] / (df['n_tokens_content'] + 1) #글 길이 대비 영상 수
    df['link_density']    = df['num_hrefs']  / (df['n_tokens_content'] + 1) #글 길이 대비 링크 수
    df['self_ref_range']  = df['self_reference_max_shares'] - df['self_reference_min_shares']
    ch_cols = [c for c in df.columns if c.startswith('data_channel_')]
    df['has_channel']     = df[ch_cols].max(axis=1) #채널 여부 → 기사가 특정 채널(테크/라이프스타일 등)에
    df['abs_sentiment']   = df['global_sentiment_polarity'].abs() #감성 관련 → 감성이 긍정이든 부정이든
    df['pos_neg_ratio']   = (df['rate_positive_words'] + 1) / (df['rate_negative_words'] + 1)
    df['kw_range']        = df['kw_max_max'] - df['kw_min_min'] # → 키워드 인기도의 편차. 범위가 클수록
    df['kw_sentiment']    = df['kw_avg_avg'] * df['global_subjectivity'] # → 키워드 인기 × 주관성.
    df['kw_pos']          = df['kw_avg_avg'] * df['rate_positive_words'] # → 키워드 인기 × 긍정 단
    df['title_subj_len']  = df['n_tokens_title'] * df['title_subjectivity'] # → 제목 길이 × 제목
    df['total_media']     = df['num_imgs'] + df['num_videos'] # → 이미지 + 영상 합계. 미디어 풍부한 글
    df['log_total_media'] = np.log1p(df['total_media']) # → total_media의 log 변환 (1개 vs 2개보다
    wd_cols = [c for c in df.columns if c.startswith('weekday_is_')]
    sat_col = [c for c in wd_cols if 'saturday' in c]
    sun_col = [c for c in wd_cols if 'sunday' in c]
    if sat_col and sun_col:
        df['is_weekend'] = df[[sat_col[0], sun_col[0]]].max(axis=1) # → 토/일 게시 여부. 주말 기사가
    df['kw_quality']       = df['kw_avg_avg'] / (df['kw_avg_min'].clip(lower=0) + 1) # → 키워드
    df['content_density']  = df['n_unique_tokens'] / (df['n_tokens_content'] + 1) # → 전체 단어
    df['title_body_ratio'] = df['n_tokens_title']  / (df['n_tokens_content'] + 1) # → 제목 길이
    log_cols = [ # ── log 변환 피처 14개 (분포가 오른쪽으로 치우친 컬럼들) <- log1p(x) = log(x+1). 1개와 1
        'n_tokens_content','num_hrefs','num_self_hrefs',
        'num_imgs','num_videos','num_keywords',
        'kw_max_max','kw_avg_max','kw_max_avg','kw_avg_avg',
        'self_reference_min_shares','self_reference_max_shares',
        'self_reference_avg_sharess','kw_range'
    ]
    for col in log_cols:
        if col in df.columns:
            df[f'log_{col}'] = np.log1p(df[col].clip(lower=0))
    df = df.replace([np.inf, -np.inf], np.nan)
    df = df.fillna(0)
    return df
```

-콘텐츠 구조 반영 변수

- img_per_word, link_density

-감성 및 키워드 결합 변수

- kw_sentiment, pos_neg_ratio

-콘텐츠 품질 및 다양성 변수

- content_density, kw_quality

-일부 파생 변수는 성능 기여도 낮아 제외

사용한 모델 3가지



LightGBM

학습이 빠름
메모리 효율이 높아
대용량 데이터에 강함



XGBoost

정밀한 최적화 수행
GPU 가속



CatBoost

하이퍼파라미터에 덜 민감
안정적인 일반화 성능
과적합 위험도 낮음

LighGBM

```python
def lgb_objective(trial):
    params = {
        'objective':         'binary',
        'metric':            'average_precision',
        'num_leaves':         trial.suggest_int('num_leaves', 31, 255),
        'learning_rate':      trial.suggest_float('learning_rate', 0.01, 0.1, log=True),
        'feature_fraction':   trial.suggest_float('feature_fraction', 0.5, 1.0),
        'bagging_fraction':   trial.suggest_float('bagging_fraction', 0.5, 1.0),
        'bagging_freq':       5,
        'min_child_samples': trial.suggest_int('min_child_samples', 5, 50),
        'lambda_l1':          trial.suggest_float('lambda_l1', 0, 5),
        'lambda_l2':          trial.suggest_float('lambda_l2', 0, 5),
        'n_estimators':       2000,
        'verbosity':         -1,
    }
    scores = []
    for tr_idx, val_idx in skf3.split(X, y):
        m = lgb.LGBMClassifier(**params)
        m.fit(X.iloc[tr_idx], y.iloc[tr_idx],
              eval_set=[(X.iloc[val_idx], y.iloc[val_idx])],
              callbacks=[lgb.early_stopping(50, verbose=False)])
        scores.append(average_precision_score(
            y.iloc[val_idx], m.predict_proba(X.iloc[val_idx])[:, 1]))
    return np.mean(scores)

lgb_study = optuna.create_study(direction='maximize')
lgb_study.optimize(lgb_objective, n_trials=100, show_progress_bar=True)
print(f"LGB 최적 PR-AUC: {lgb_study.best_value:.4f}")
```

파라미터	탐색 범위
num_leaves	31 ~ 255
learning_rate	0.01 ~ 0.1
feature/bagging_fraction	0.5 ~ 1.0
min_child_samples	5 ~ 50
lambda_l1 / l2	0 ~ 5

Optuna 100회 탐색 / 3-Fold / 최적 PR-AUC 0.3287

본학습: n_estimators 6000 + Early Stopping 200회

LighGBM

```python
# LightGBM 5-Fold
print("\n LightGBM 학습...")
best_lgb = lgb_study.best_params
best_lgb.update({'objective':'binary','metric':'average_precision',
                 'bagging_freq':5,'n_estimators':6000,'verbosity':-1})

oof_lgb=np.zeros(len(X)); lb_preds_lgb=np.zeros(len(X_lb)); test_preds_lgb=np.zeros(len(X_test))
for fold,(tr_idx,val_idx) in enumerate(skf.split(X,y)):
    X_tr,X_val=X.iloc[tr_idx],X.iloc[val_idx]; y_tr,y_val=y.iloc[tr_idx],y.iloc[val_idx]
    m=lgb.LGBMClassifier(**best_lgb)
    m.fit(X_tr,y_tr,eval_set=[(X_val,y_val)],
          callbacks=[lgb.early_stopping(200,verbose=False),lgb.log_evaluation(999)])
    oof_lgb[val_idx]=m.predict_proba(X_val)[:,1]
    lb_preds_lgb+=m.predict_proba(X_lb)[:,1]/5
    test_preds_lgb+=m.predict_proba(X_test)[:,1]/5
    print(f"  LGB Fold {fold+1}: {average_precision_score(y_val,oof_lgb[val_idx]):.4f}")
print(f"✅ LGB OOF: {average_precision_score(y,oof_lgb):.4f}")
```

최종 OOF PR-AUC ->0.3222

```
🚀 LightGBM 학습...
 LGB Fold 1: 0.2983
 LGB Fold 2: 0.3379
 LGB Fold 3: 0.3287
 LGB Fold 4: 0.3212
 LGB Fold 5: 0.3403
✅ LGB OOF: 0.3222
```

XGBoost

```python
def xgb_objective(trial):
    params = {
        'objective':             'binary:logistic',
        'eval_metric':           'aucpr',
        'max_depth':              trial.suggest_int('max_depth', 4, 8),
        'learning_rate':          trial.suggest_float('learning_rate', 0.01, 0.1, log=True),
        'n_estimators':           2000,
        'subsample':              trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree':       trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'min_child_weight':       trial.suggest_int('min_child_weight', 5, 30),
        'gamma':                  trial.suggest_float('gamma', 0, 5),
        'reg_alpha':              trial.suggest_float('reg_alpha', 0, 5),
        'reg_lambda':             trial.suggest_float('reg_lambda', 0, 5),
        'verbosity':              0,
        'random_state':           42,
        'early_stopping_rounds':  50,
        'device':                 'cuda',
        'tree_method':            'hist',
    }
    scores = []
    for tr_idx, val_idx in skf3.split(X, y):
        m = XGBClassifier(**params)
        m.fit(X.iloc[tr_idx], y.iloc[tr_idx],
              eval_set=[(X.iloc[val_idx], y.iloc[val_idx])],
              verbose=False)
        scores.append(average_precision_score(
            y.iloc[val_idx], m.predict_proba(X.iloc[val_idx])[:, 1]))
    return np.mean(scores)

xgb_study = optuna.create_study(direction='maximize')
xgb_study.optimize(xgb_objective, n_trials=100, show_progress_bar=True)
print(f"XGB 최적 PR-AUC: {xgb_study.best_value:.4f}")
```

파라미터	탐색 범위
max_depth	4~8
learning_rate	0.01 ~ 0.1
subsample / colsample_bytree	0.5 ~ 1.0
min_child_weight	5 ~ 30
gamma / reg_alpha / lambda	0 ~ 5

Optuna 100회 탐색 / 3-Fold / 최적 PR-AUC 0.3292

본학습: n_estimators 6000 + Early Stopping 200회

XGBoost

```python
# XGBoost 5-Fold (gpu 사용)
print("\n XGBoost 학습...")
best_xgb = xgb_study.best_params
best_xgb.update({'objective':'binary:logistic','eval_metric':'aucpr',
                 'n_estimators':6000,'verbosity':0,'random_state':42,
                 'early_stopping_rounds':200,'device':'cuda','tree_method':'hist'})

oof_xgb=np.zeros(len(X)); lb_preds_xgb=np.zeros(len(X_lb)); test_preds_xgb=np.zeros(len(X_test))
for fold,(tr_idx,val_idx) in enumerate(skf.split(X,y)):
    X_tr,X_val=X.iloc[tr_idx],X.iloc[val_idx]; y_tr,y_val=y.iloc[tr_idx],y.iloc[val_idx]
    m=XGBClassifier(**best_xgb)
    m.fit(X_tr,y_tr,eval_set=[(X_val,y_val)],verbose=False)
    oof_xgb[val_idx]=m.predict_proba(X_val)[:,1]
    lb_preds_xgb+=m.predict_proba(X_lb)[:,1]/5
    test_preds_xgb+=m.predict_proba(X_test)[:,1]/5
    print(f"  XGB Fold {fold+1}: {average_precision_score(y_val,oof_xgb[val_idx]):.4f}")
print(f"✅ XGB OOF: {average_precision_score(y,oof_xgb):.4f}")
```

최종 OOF PR-AUC → 0.3271

```
🚀 XGBoost 학습...
 XGB Fold 1: 0.3061
 XGB Fold 2: 0.3466
 XGB Fold 3: 0.3374
 XGB Fold 4: 0.3210
 XGB Fold 5: 0.3421
✅ XGB OOF: 0.3271
```

CatBoost

```python
# Optuna CatBoost tuning
print("\n Optuna CAT 튜닝 중...")
def cat_objective(trial):
    params = {
        'iterations':          2000,
        'learning_rate':       trial.suggest_float('learning_rate', 0.01, 0.1, log=True),
        'depth':               trial.suggest_int('depth', 4, 8),
        'l2_leaf_reg':         trial.suggest_float('l2_leaf_reg', 1, 10),
        'bagging_temperature': trial.suggest_float('bagging_temperature', 0, 1),
        'random_strength':     trial.suggest_float('random_strength', 0, 3),
        'min_data_in_leaf':    trial.suggest_int('min_data_in_leaf', 1, 30),
        'eval_metric':         'PRAUC',
        'random_seed':         42,
        'verbose':             0,
    }
    scores = []
    for tr_idx, val_idx in skf3.split(X, y):
        m = CatBoostClassifier(**params)
        m.fit(X.iloc[tr_idx], y.iloc[tr_idx],
              eval_set=(X.iloc[val_idx], y.iloc[val_idx]),
              early_stopping_rounds=50)
        scores.append(average_precision_score(
            y.iloc[val_idx], m.predict_proba(X.iloc[val_idx])[:, 1]))
    return np.mean(scores)

cat_study = optuna.create_study(direction='maximize')
cat_study.optimize(cat_objective, n_trials=60, show_progress_bar=True)
print(f"CAT 최적 PR-AUC: {cat_study.best_value:.4f}")
```

파라미터	탐색 범위
depth	4~8
learning_rate	0.01 ~ 0.1
l2_leaf_reg	1~10
random_strength	0~3
min_data_in_leaf	1~30

최적 튜닝 PR-AUC: 0.3324

탐색 횟수 60회 (1회당 학습 시간이 길어 조정)

n_estimators: 6000 + Early Stopping 200회

CatBoost

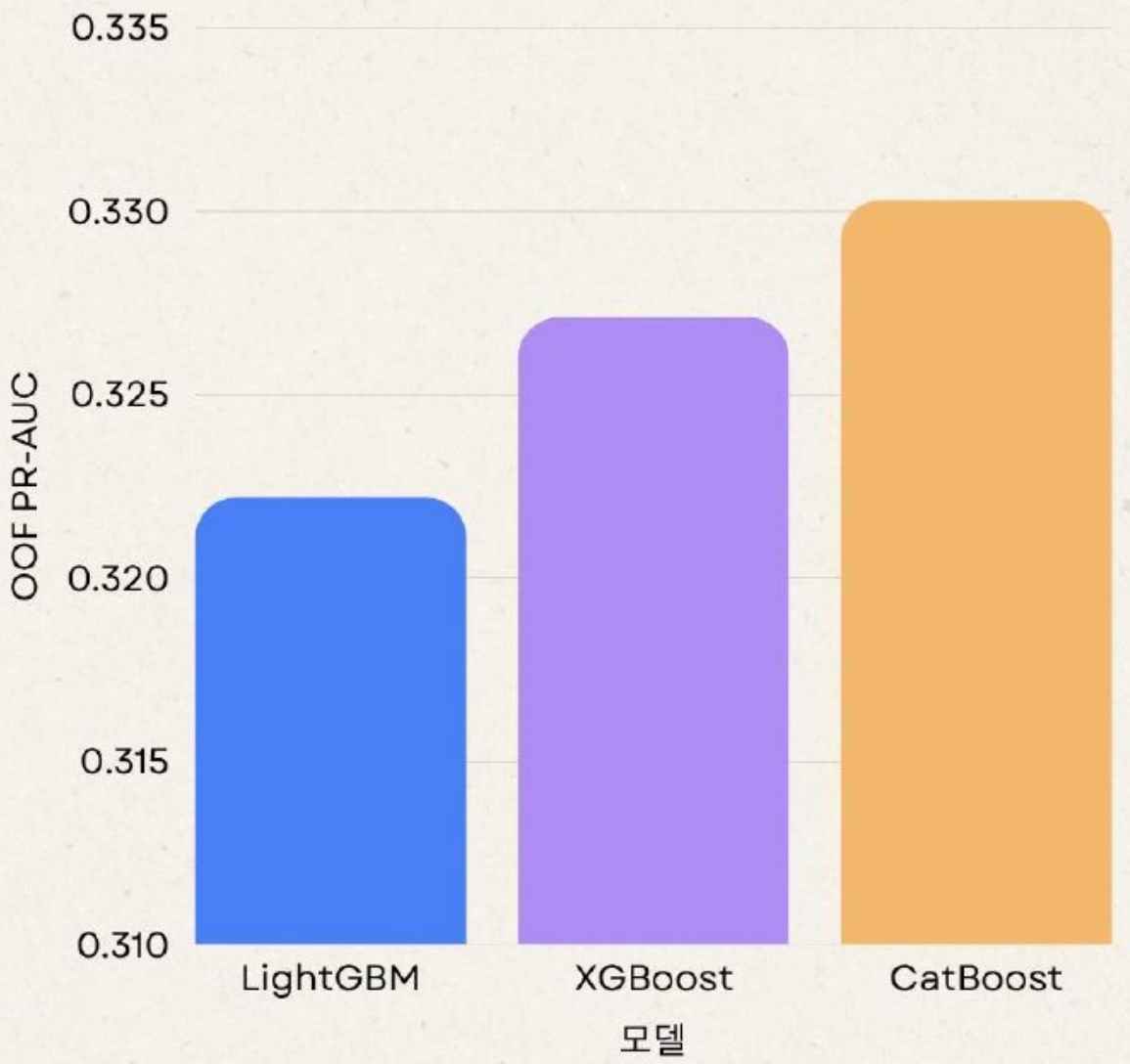
```python
# CatBoost 5-Fold
print("\n CatBoost 학습...")
best_cat = cat_study.best_params
best_cat.update({'iterations':6000,'eval_metric':'PRAUC','random_seed':42,'verbose':0})

oof_cat=np.zeros(len(X)); lb_preds_cat=np.zeros(len(X_lb)); test_preds_cat=np.zeros(len(X_test))
for fold,(tr_idx,val_idx) in enumerate(skf.split(X,y)):
    X_tr,X_val=X.iloc[tr_idx],X.iloc[val_idx]; y_tr,y_val=y.iloc[tr_idx],y.iloc[val_idx]
    cat=CatBoostClassifier(**best_cat)
    cat.fit(X_tr,y_tr,eval_set=(X_val,y_val),early_stopping_rounds=200)
    oof_cat[val_idx]=cat.predict_proba(X_val)[:,1]
    lb_preds_cat+=cat.predict_proba(X_lb)[:,1]/5
    test_preds_cat+=cat.predict_proba(X_test)[:,1]/5
    print(f"  CAT Fold {fold+1}: {average_precision_score(y_val,oof_cat[val_idx]):.4
print(f"✅ CAT OOF: {average_precision_score(y,oof_cat):.4f}")
```

최종 OOF PR-AUC -> 0.3303

```
🚀 CatBoost 학습...
CAT Fold 1: 0.3095
CAT Fold 2: 0.3526
CAT Fold 3: 0.3268
CAT Fold 4: 0.3391
CAT Fold 5: 0.3426
✅ CAT OOF: 0.3303
```

성능 비교



세 모델 중에서는 CatBoost가 가장 높은 성능을 보임

앙상블

단일 모델의 성능을 더욱 향상시키기 위해 앙상블 기법을 적용
단순 평균, 가중치 앙상블, 스태킹 세 가지 방식

```python
# 스태킹 앙상블
print("\n 스태킹 앙상블 학습 중...")
meta_train=np.column_stack([oof_lgb,oof_xgb,oof_cat])
meta_lb   =np.column_stack([lb_preds_lgb,lb_preds_xgb,lb_preds_cat])
meta_test =np.column_stack([test_preds_lgb,test_preds_xgb,test_preds_cat])
scaler=StandardScaler()
meta_train_s=scaler.fit_transform(meta_train)
meta_lb_s   =scaler.transform(meta_lb)
meta_test_s =scaler.transform(meta_test)
oof_stacked=np.zeros(len(X))
for tr_idx,val_idx in skf.split(meta_train_s,y):
    mm=LogisticRegression(C=0.1,max_iter=1000)
    mm.fit(meta_train_s[tr_idx],y.iloc[tr_idx])
    oof_stacked[val_idx]=mm.predict_proba(meta_train_s[val_idx])[:,1]
mm_final=LogisticRegression(C=0.1,max_iter=1000)
mm_final.fit(meta_train_s,y)
lb_stacked  =mm_final.predict_proba(meta_lb_s)[:,1]
test_stacked=mm_final.predict_proba(meta_test_s)[:,1]
print(f"스태킹 OOF PR-AUC: {average_precision_score(y,oof_stacked):.4f}")
```

```python
# 최종 앙상블 자동 선택 및 결과물 저장
score_simple  =average_precision_score(y,(oof_lgb+oof_xgb+oof_cat)/3)
score_weighted=average_precision_score(y,oof_weighted)
score_stacked =average_precision_score(y,oof_stacked)
print(f"\n 앙상블 방법별 OOF PR-AUC 비교:")
print(f"  단순 평균     : {score_simple:.4f}")
print(f"  가중치 최적화: {score_weighted:.4f}")
print(f"  스태킹        : {score_stacked:.4f}")
scores_dict={
    'weighted':(score_weighted,lb_weighted,   test_weighted),
    'stacked': (score_stacked, lb_stacked,    test_stacked),
    'simple':  (score_simple,
                (lb_preds_lgb+lb_preds_xgb+lb_preds_cat)/3,
                (test_preds_lgb+test_preds_xgb+test_preds_cat)/3),
}
best_name=max(scores_dict,key=lambda k:scores_dict[k][0])
_,lb_final,test_final=scores_dict[best_name]
print(f"\n 최종 선택: {best_name} ({scores_dict[best_name][0]:.4f})")
```

앙상블




```
🔍 앙상블 가중치 최적화 중...
✅ 최적 가중치 → LGB:0.019 | XGB:0.337 |
CAT:0.644
✅ 가중치 앙상블 OOF PR-AUC: 0.3320

🔍 스태킹 앙상블 학습 중...
스태킹 OOF PR-AUC: 0.3301

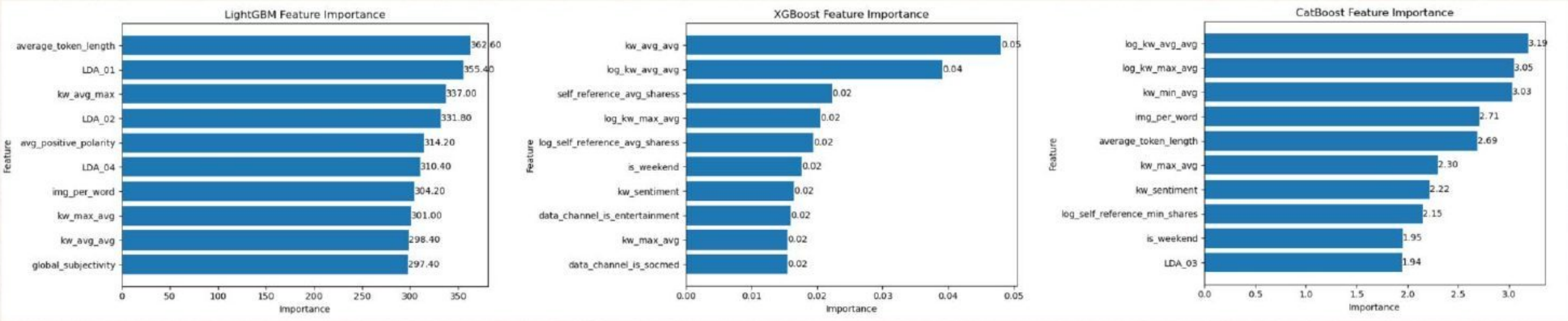
앙상블 방법별 OOF PR-AUC 비교:
  단순 평균     : 0.3305
  가중치 최적화: 0.3320
  스태킹        : 0.3301

최종 선택: weighted (0.3320)
```

weighted assemble 선택됨

결과 해석

각 모델에서 제공하는 Feature Importance 함수
(LightGBM/XGBoost: feature_importances_,
CatBoost: get_feature_importance())를 사용하여 중요도 계산



키워드 인기도 → kw_avg_avg, kw_max_avg, log_kw_avg_avg

콘텐츠 구조 → img_per_word, average_token_length

감성 정보 → kw_sentiment

과거 인기 패턴 → self_reference_avg_shares, log_self_reference_min_shares

결과 해석

모델 결과를 통해 얻은 인사이트



feature engineering

“원본 데이터를 그대로 쓰지 않는다.”

modeling

“하이퍼파라미터는 탐색하여 결정한다.”

Ensembling

“하나보다는 여럿이 낫다, 단 올바른 방식으로“

Whole Project

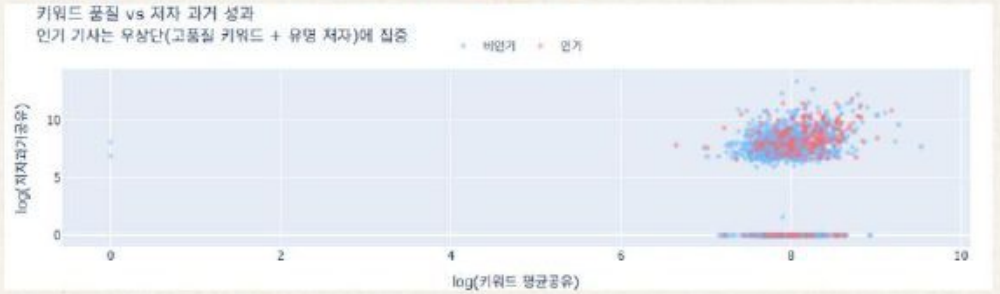
"좋은 피처 × 올바른 튜닝 × 다양한 모델 = 앙상블 시너지"

단계 하나하나가 독립적이지 않고, 앞 단계 품질이 뒷 단계 성능을 결정한다

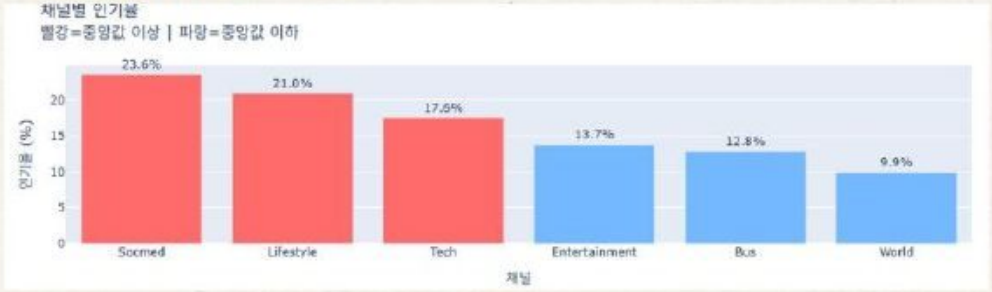
결과 해석

인기 기사와 관련 있어 보이는 패턴

1 키워드 품질이 높을 수록 인기
2 저자의 과거 기사 성과가 강한 신호



3 채널(카테고리) 효과



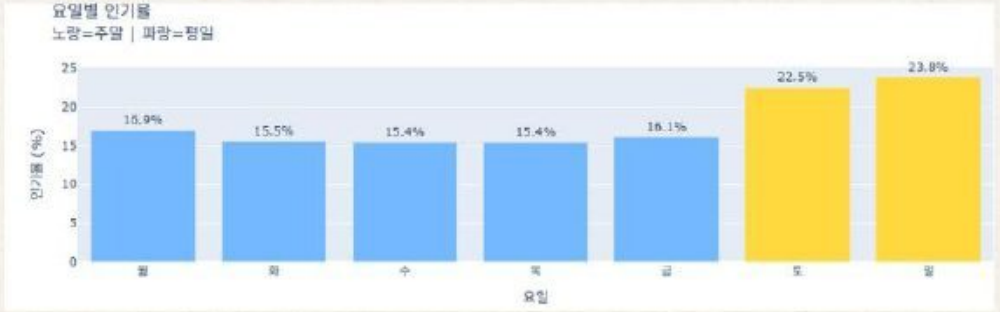
4 미디어 복합 사용



5 콘텐츠 길이의 역 U자 관계



6 평일보다 주말 게시



7 감성의 방향보다 강도



한계 및 해결



- 하이퍼 파라미터 튜닝시간이 길다
- 과적합
- 모델 최적화의 어려움



- 3-Fold + Early Stopping으로 튜닝 시간 단축
- Early Stopping + 정규화로 과적합 방지
- Optuna 자동 탐색으로 최적 파라미터 조합

+ 피처 수 축소 (중요도 낮은 피처 제거)
+ 새로운 모델 추가를 통한 다채로운 학습